Università degli Studi di Trento Corso di Programmazione I – E3: 145430/145935 5° Appello – 01/09/2022 – a.a. 2021/2022	**PC N°**
COGNOME ..... **NOME** ..... **MATRICOLA**	
RITIRATO	**FIRMA PER RITIRARSI** .....

NON è consentito utilizzo di alcun strumento elettronico, informatico etc. durante la prova.
Fa parte della valutazione la leggibilità *del codice C/C++.*
Tempo a disposizione: 100 minuti.

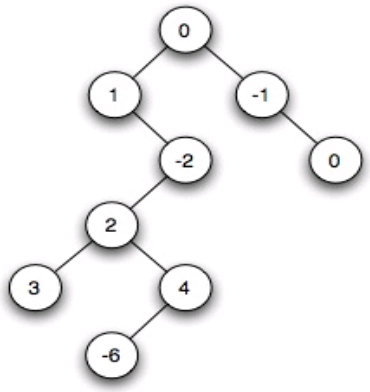
PARTE TEORICA: 12 PUNTI
Tempo a disposizione consigliato: 30 minuti.

[Domanda1 – 3 punti]
[A] Si indichi la differenza tra la codifica per un numero reale in virgola fissa e virgola mobile.
[B] Si mostri (dettagliando ogni passaggio) per la codifica in virgola fissa cosa vale in decimale $[0101.0100]_2$.
[C] Si mostri (dettagliando ogni passaggio) per la codifica in virgola mobile cosa vale in decimale la sequenza binaria:

1	11111111	10000000000000000000000000000000

[Domanda2 – 3 punti]
Per l'albero binario in figura (NON ORDINATO) calcolare:

Cammino									
Altezza									
Visita pre-ordine									
Visita post-ordine									
Visita in-ordine									
Visita level-ordine									



[Domanda3 – 5 punti]
Data la funzione di ordinamento **mysort**

NOTA la funzione **swap** scambia i valori

```
void mysort(int dati[], int dim) {
  for (int i=0; i < dim-1; i++) {
    int min = i;
    for(int j=i+1; j<dim; j++) {
     if (dati[j] < dati[min]) { min = j; }
    }
    swap(dati[i], dati[min]);
  }
}
```

[A] applicarla a questo insieme di dati

6	1	0	8	4	3

Riportando nella tabella la situazione dell'array **dati** dopo l'operazione di swap.

i=0						
i=1						
i=2						
i=3						
i=4						
i=5						

[B] riscrivere l'algoritmo per invertire l'ordinamento dell'array

[Domanda4 – 1 punti]
Dato il seguente frammento di codice:

```
int* myfunz (int dim) {
  int* punt;
  punt = new int[dim];
  for (int i=0; i < dim; ++i) { punt[i]=i; }
  return punt;
}
```

La memoria allocata all'interno della funzione **myfunz** mediante l'operatore new:

SI ☐ NO ☐ viene deallocata quando il controllo passa all'esterno della funzione **myfunz**

SI ☐ NO ☐ non può più venire deallocata poiché l'operatore delete non compare all'interno della funzione stessa

SI ☐ NO ☐ rimane allocata fin quando il main non termina o finché non viene esplicitamente deallocata tramite l'istruzione delete

SI ☐ NO ☐ rimane allocata fin quando il main non termina o finché non venga esplicitamente deallocata tramite una istruzione delete [ ]

PARTE PRATICA: 21 PUNTI
Tempo a disposizione consigliato: 70 minuti.

[A - punti 7] Inserire i seguenti tipi in un file ***dati.h*** ed implementarlo in un file ***dati.cpp***

```
typedef enum Tmail {PEC, STD} Tmail;
typedef struct Tdata{ int gg,mm,aa; } Tdata;
typedef struct Tmessaggio{
  int id;
  Tmail tipo;
  Tdata data;
  char messaggio[100];
  Tmessaggio () { /*  da implementare */ }
  void stampa() { /*  da implementare → formato: vedi punto D */ }
} Tmessaggio;
typedef struct Tmailbox {
  Tmessaggio msg;
  Tmailbox* next;
  Tmailbox ()   { /*  da implementare */ }
  void stampa() { /*  da implementare */ }
} Tmailbox; // LIFO realizzato con lista semplicemente concatenata
```

[B - punti 4] Modificare il file ***main.cpp*** includendo il file ***dati.h*** ed inserendo il seguente codice nel ***main***:

```
Tmailbox* box[2];
/* inizializzare le 2 LIFO in modo opportuno */
/* inserire 10 elementi di tipo Tmessaggio (usare la funzione newMsg)
  nelle 2 LIFO: se tipo==PEC inserire in box[0] altrimenti box[1] (usare
  la funzione addMsg)
  NOTA per utilizzo funzione newMsg: id deve essere progressivo ed univoco
*/
stampaMailbox(box,2);
saveMailbox(box, 2);
```

[C - punti 3] Implementare una funzione **`newMsg`** che riceve in input l'indirizzo di una variabile di tipo **`Tmessaggio`** ed un valore **`int id`** e la inizializza con:

- Campo **`id`** con il valore passato;
- campo **`tipo`** valore casuale tra PEC e STD;
- campo **`data`** inizializzato con: aa=2022, gg valore casuale tra 1 e 31, mm valore casuale tra 1 e 12;
- campo **`messaggio`** letto da tastiera.

[D - punti 3] Implementare una funzione **`addMsg`** che riceve come input un **`Tmailbox* mb`**, ed una variabile **`m`** di tipo **`Tmessaggio.`** La funzione inserisce tale valore nella lista LIFO **`mb`** (modalità **`insertLast`**) e ritorna la lista aggiornata.

[E - punti 2] Implementare una funzione **`stampaMailbox`** che riceve in input un array di tipo **`Tmailbox* mba`** e la sua dimensione e stampa il contenuto delle `LIFO` utilizzando opportunamente il metodo **`stampa`** di **`Tmailbox/Tmessaggio`**. Nella forma:

`IDMAIL=``id` **`tipo=`**`tipo` **`data=`**`gg/mm/aa`

NOTA: per enumerativo usare uno switch case

[F - punti 2] Implementare una funzione **`saveMailbox`** che riceve in input un array di tipo **`Tmailbox* mba`** e la sua dimensione e per ogni mail presente nelle LIFO, inserisce in un file denominato "**`OLDMail.txt`**" se il campo data < 01/06/2022 nella forma:

`IDMAIL=``id` **`msg=`** `messaggio`